Project 1.

ACME Vet clinic has 10 doctors and 20 technicians and has hired you to create a web application to help them with scheduling. The clinic is a 24 hour clinic. Each doctor and each technician typically work four shifts every week and they need at least 10 hours break between two shifts. Additional constraints are that no doctor should have two surgery days or two overnight shifts in the same week. Here are the possible shifts for doctors:

M T W Th F S:

7.30am-6.30pm (Early Morning)

8.30am-7.30pm (8.30 Shift)

9.30am-8.30pm (Late Doctor)

Sunday:

8am – 8pm (Sunday Shift)

Overnight shift:

8pm – 8 am (Overnight)

Usually, there is at least one doctor that takes each shift every day. In addition, Monday-Saturday there is a surgeon who works 7.30am-6.30pm. If we assume there is one doctor per shift, which makes five shifts (including the surgery shift) everyday Monday-Saturday and two shifts on Sunday, there will be a total of 32 shifts. Since each doctor works four shifts, eight doctors can cover all the shifts in the practice. Since there are 10 doctors, the extra eight shifts are distributed throughout the week, where having an extra doctor for the Saturday Early Morning shift and Friday Late Doctor shift is a priority. Other than that, the preference is to allocate the remaining shifts as Early Morning Shifts. There are never two doctors overnight or on Sunday.

Technicians have the same shifts as doctors, where every doctor should have at least one technician working with them. There should be only one technician on overnights and two technicians on Sundays. The rest of the scheduling for the technicians should be spread as evenly as possible.

The system should allow each doctor and each technician to login and check their schedule for the next two weeks. It will be nice to present the information in tabular format. The schedule should show the shifts and surgery shifts (for doctors) should be displayed in a different color. Doctors should also be able to see the technicians that they will work with and technicians should be able to see the doctors that they are working with.

The system should also allow for a system administrator who can create/remove doctors and technicians and set their passwords. The system administrator will be the only one with this privilege. Their login is “admin”, password “admin”, and they should be forced to change their password the first time they login.

The system should also allow for doctors and technicians to request the dates and time they work or to take time off. Every doctor and technician is allowed 8 days of vacation and 4 sick days during a calendar year. There should be a way for the doctors/technicians to see how much vacation and sick time they

have used. Requests are processed on first-come first-serve basis. In other words, if a request is possible, it is satisfied and the schedule is rearranged accordingly. If a request conflicts with existing requests, it is denied. The system should be able to immediately say if a request is granted or denied. If a request is denied, the system needs to explain the reason. Possible reasons include: “request is too late”, “you don’t have any more vacation time”, “you don’t have any more sick days”, “we will be short on doctors if we grant your request”, “your request cannot be granted because it conflicts previous shift requests that were granted”. A request for vacation needs to be made at least three weeks in advance. A request for a sick day cannot be for today, but it can be for any other day. The system should keep track of the amount of vacation and sick days of everyone.

When done, zip the project and submit it on Canvas (only one member of the team needs to submit the project). Possible technologies to use are as follows.

- Java Server Faces and PostgreSQL as the backend.

- React as the front end (feel free to use AI to generate front end code) and Java Spring Boot with PostgreSQL as the back end. Make sure to write the embedded queries yourself and not use libraries, such as Spring DATA JPA and JdbcTemplates.

- Stand-alone application using Java Swing and PostgreSQL as the backend.

- Stand-alone application using Java FX and PostgreSQL as the backend. Feel free to use Gluon Scene Builder.

Please additionally create the following files:

- UserManual <- User Manual of how to use the system, including initial logins/passwords.
- LogFile <- Who did what and how much time did it took.
- Assumptions <- Any assumptions you have made. Please talk to me during lab times to make sure your assumptions are valid.
- ER diagram of the software.
- All the create table commands that were generated from the ER diagram.

The project should be done in teams of three or four. Teams of one or two are allowed only with my approval.

Very important: the requirements are vague on purpose. Please talk with the client (me) to iron out the requirements. This is a project in software engineering as much as a database project.